Documentación – Comunicación inalámbrica con NRF24L01 (ESP32 con ESP32/Arduino)

Fecha: 06/03/2026

Integrantes:

• Eduardo Cadengo López

• Itzel Citlalli Martell De La Cruz

• Damian Alexander Diaz Piña

Contenido

Introducción: 1
Materiales utilizados: 1
Conexiones: 2
Lógica del programa: 3
Código: 3
Procedimiento: 5
Evidencias 6
Troubleshooting (Problemas comunes): 6
Video de funcionamiento verificado por el profesor: https://youtu.be/Wdm9OlAvnIU 7
GitHub: 7
Conclusiones 7

Introducción:

En esta práctica se implementa la comunicación inalámbrica punto a punto utilizando el módulo NRF24L01 para establecer un enlace entre dos microcontroladores, configurando uno como transmisor (TX) y el otro como receptor (RX). El objetivo se cumple cuando, al monitorear la terminal del RX, se observe un mensaje indicando que la comunicación se ha establecido y que, mientras el TX permanezca energizado, el RX muestre periódicamente el texto recibido.

Materiales utilizados:

- 2 Módulos NRF24L01
- 2 ESP32 o Arduino

- Jumpers de hembra a hembra para conectarlos del esp32 a la antena
- 2 Cables para conectar el Esp32 o el Arduino a la computadora, dependerá de la entrada de tu placa, puede ser tipo C, micro USB, etc.

Conexiones:

-Pines NRF24L01

- GND - GND
- VCC - 3.3 V (no 5 V)
- CE - pin configurable (en el ejemplo GPIO 4)
- CSN (CS) - pin SPI SS (en el ejemplo GPIO 5)
- SCK - reloj SPI
- MOSI - datos SPI (salida MCU → NRF)
- MISO - datos SPI (entrada MCU ← NRF)
- IRQ - (opcional; no usado en esta práctica)

ESP32 Conexiones, es lo mismo para el transmisor que para el receptor

Señal (Antena)	Pin ESP32
CE	**GPIO 4**
CSN	**GPIO 5**
SCK	**GPIO 18**
MOSI	**GPIO 23**
MISO	**GPIO 19**
VCC	**3.3 V**
GND	**GND**

Arduino UNO (si usas UNO en RX o TX. Nosotros no usamos Arduino, pero de igual manera aquí están las conexiones por si se llegara a utilizar)

Señal	Pin UNO
CE	**7** (ejemplo)
CSN	**8** (ejemplo)
SCK	**13**
MOSI	**11**
MISO	**12**
VCC	**3.3 V** (no 5 V)
GND	**GND**

Lógica del programa:

Inicialización del radio: configurar canal, potencia y dirección (“pipe”) de transmisión/recepción.

TX(Transmisor): deja de escuchar (stopListening) y envía periódicamente el mensaje (“Hola”).

RX(Recepetor): arranca en modo escucha (startListening) y, cuando hay datos (radio.available()), lee el payload y lo imprime por Serial.

Criterio de cumplimiento: mientras el TX esté encendido, el RX debe seguir recibiendo y mostrando el mensaje periódicamente.

Código:

CODIGO DEL TRASNMISOR:

```
#include <SPI.h>

#include <RF24.h>

#define CE_PIN 4
#define CSN_PIN 5

RF24 radio(CE_PIN, CSN_PIN);
```

```
// Dirección de 5 bytes + '\0' en tu ejemplo; RF24 usa 5 bytes efectivos.
// Mantén la misma entre TX y RX.
const byte direccion[6] = "00001";

void setup() {
  Serial.begin(115200);
  if (!radio.begin()) {
    Serial.println("NRF24 no detectado o no esta bien conectado");
    while (1);
  }

  radio.setChannel(67);         // Canal 67 (debe coincidir con el RX)
  radio.setPALevel(RF24_PA_LOW); // Potencia baja (ajusta a MEDIUM/HIGH si
hay fallos)
  // radio.setDataRate(RF24_250KBPS); // Opcional: mejor
alcance/fiabilidad

  radio.openWritingPipe(direccion);
  radio.stopListening();         // Modo TX
}

void loop() {
  const char texto[] = "Hola";
  bool ok = radio.write(&texto, sizeof(texto));
  if (ok) {
    Serial.println("Mensaje enviado");
  } else {
    Serial.println("Error al enviar");
  }
  delay(1000); // 1 segundo
}
```

CODIGO DEL RECEPTOR:

```
#include <SPI.h>

#include <RF24.h>

#define CE_PIN 4
#define CSN_PIN 5

RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

```
void setup() {
  Serial.begin(115200);
  if (!radio.begin()) {
    Serial.println("NRF24 no detectado o no esta bien conectado");
    while (1);
  }

  radio.setChannel(67);          // Mismo canal
  radio.setPALevel(RF24_PA_LOW); // Ajustable según alcance/ruido
  // radio.setDataRate(RF24_250KBPS); // Opcional: mejor alcance

  radio.openReadingPipe(0, direccion);
  radio.startListening();        // Modo RX
  Serial.println("RX listo. Esperando mensajes...");
}

void loop() {
  if (radio.available()) {
    char texto[32] = {0}; // payload max 32 bytes
    radio.read(&texto, sizeof(texto));
    Serial.print("Recibido el mensaje: ");
    Serial.println(texto);
  }
}
``
```

Procedimiento:

1.- Cableado

Conecta ambos NRF24L01 (TX y RX) a sus microcontroladores según las tablas.

Añade capacitor de 10–47 µF entre 3.3 V y GND del módulo (muy recomendado).

2.- Cargar el código

Sube el sketch de TX al primer dispositivo.

Sube el sketch de RX al segundo dispositivo.

3.- Monitoreo para ver si funciono

Abre el Monitor Serial del RX a 115200, importante seleccionar 115200, porque si no, el mensaje no cargara adecuadamente en el monitor serial.

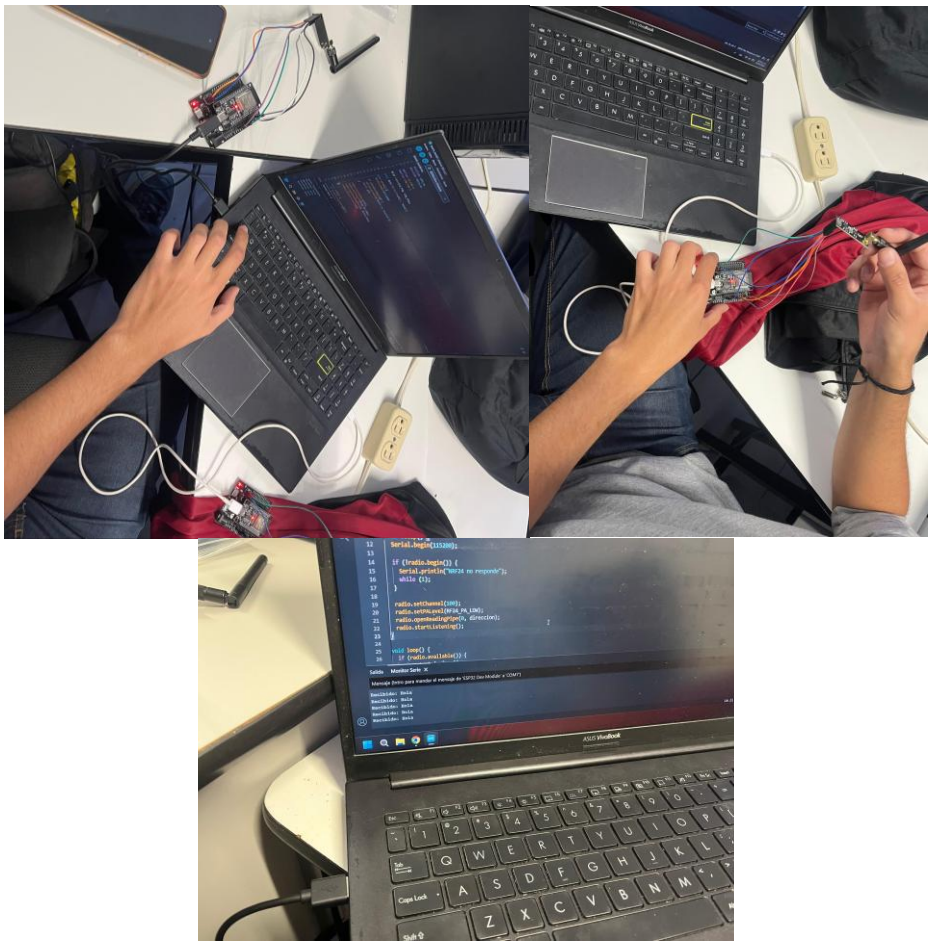
Energiza el TX y observa en el RX:

<u>RX listo. Esperando mensajes...</u>

Recibido el mensaje: Hola

Recibido el mensaje: Hola

Evidencias



Troubleshooting (Problemas comunes):

No detecta el NRF24L01 (radio.begin() falla):

- Verifica VCC=3.3 V y GND.
- Revisa CE/CSN, SCK/MOSI/MISO según tu placa.

No recibe nada:

- Asegúrate de que canal (setChannel) y dirección (openWritingPipe/openReadingPipe) coincidan en TX y RX.

- Intenta bajar data rate a 250 kbps (radio.setDataRate(RF24_250KBPS)) y/o subir potencia (RF24_PA_HIGH) si hay distancia/obstáculos.
- Evita interferencias: cambia el canal (por ejemplo 97) si hay saturación en 2.4 GHz.

Mensajes erráticos:

- Inicializa el buffer de lectura en RX (char texto[32] = {0};).
- Evita delay muy corto en TX; 500–1000 ms es razonable para pruebas.

Alimentación de la antena:

- Nunca le des más de 3.3 de voltaje a la antena o podrás llegar a quemar la placa

Video de funcionamiento verificado por el profesor:
https://youtu.be/Wdm9OlAvnIU

GitHub:
https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/1_PracticaNFR24L01_HandShake

Conclusiones

Itzel: En esta práctica se trabajó con el ESP32 para conectarlo a una antena. Esto permitió entender mejor cómo funcionan los dispositivos que se conectan de forma inalámbrica y cómo pueden enviar o recibir información a través de esta. También se pudo observar que la antena influye en la calidad de la señal y en la estabilidad de la conexión. En general, la práctica me ayudó a comprender de forma más clara cómo se configura y utiliza el ESP32 para proyectos relacionados con las antenas inalambricas.

Eduardo: La práctica nos permitió entender mejor cómo funciona la comunicación inalámbrica con los módulos NRF24L01 y cómo deben configurarse correctamente tanto el transmisor como el receptor. Aunque el proceso fue sencillo en teoría, uno de los mayores retos fue que las conexiones físicas entre el NRF24L01 y el ESP32 no estaban bien hechas al inicio. Esto provocó que el radio no respondiera y nos hiciera pensar que había un problema en el código, cuando en realidad el problema era el cableado. Una vez que revisamos con calma y conectamos cada pin en su lugar correcto, la comunicación empezó a funcionar sin problemas y el receptor pudo mostrar el mensaje enviado de manera continua.

Damian: En esta práctica pude familiarizarme con el uso del NRF24L01 y comprender mejor cómo se establece una comunicación inalámbrica entre dos dispositivos. Aunque el funcionamiento general del módulo resultó más sencillo de lo que esperaba, uno de los retos que más tiempo nos tomó resolver fue la configuración inicial del canal y la

dirección de comunicación. Al principio ambos dispositivos estaban programados con parámetros diferentes, lo que ocasionaba que el receptor no detectara ningún mensaje y pareciera que el sistema no funcionaba. Después de revisar el código con más detalle y asegurarnos de que tanto el TX como el RX usaran exactamente el mismo canal y dirección, la comunicación comenzó a fluir sin problemas